Decidability 01

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Decidability

- A language is **decidable** if there is a **decider** decides it
- A **language** is a set of strings
- A **decider** is a Turing machine that halts on all inputs
- A Turing machine **halts** when it enters either the accept state (q_{accept}) or the reject state (q_{reject})
- To show that a language A is **decidable**:
  - Construct a TM M that you think it decides the language A and prove that
    1. For every $s \in A$, M must accept s and
    2. For every $s \notin A$, M must reject s

Decidability

- We constructed M and proved that
  1. For every $s \in A$, M must accept s and
  2. For every $s \notin A$, M must reject s
- This allows us to conclude the following:
  1. $L(M) = A$
     - M accepts all strings in A and does not accept all strings not in A
  2. M is a decider (always halts)
     - Given a string s, it is either in A or not in A
     - If $s \in A$, M accepts s (halts)
     - If $s \notin A$, M rejects s (halts)
  3. A is decidable
     - The decider M decides A

Decidable Languages

Problem

The problem of determining whether a DFA accepts a string.

- Let $\langle x \rangle$ be a string representation of the object x
- The above problem can be converted into a membership problem (language) A_{DFA}

$$A_{\text{DFA}} = \{\langle B, w \rangle \mid B \text{ is a DFA that accepts input string } w\}$$

- Now, the problem whether DFA B accepts the string w becomes the problem whether $\langle B, w \rangle$ is in A_{DFA}

Decidable Languages

- The language A_{DFA}

$$A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ is a DFA that accepts input string } w\}$$

- About the above language:
  - If F is a DFA and it accepts 01101, $\langle F, 01101\rangle$ is in A_{DFA}
    - $\langle F, 01101\rangle$ can be the formal definition of DFA F, followed by #, and followed by 01101

$$\langle F, 01101\rangle = \text{"}(\{q_0, \dots\}, \{0, 1\}, \dots)\#01101\text{"}$$

  - If G is a DFA and it rejects ε, $\langle G, \varepsilon\rangle$ is not in A_{DFA}
- Recall a set definition defined by a predicate:
  - What can we conclude if $\langle B, w\rangle \in A_{\text{DFA}}$?
    - B is a DFA that accepts input string w
  - What can we conclude if $\langle B, w\rangle \notin A_{\text{DFA}}$?
    - B is not a DFA,
    - w is not a string, or
    - B is a DFA that rejects input string w

Decidable Languages

- The language A_{DFA}

$$A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ is a DFA that accepts input string } w\}$$

- To show that A_{DFA} is decidable, we must show that there exists a decider M that decides A_{DFA}
  - Construct a TM M that we think it decides A and prove that
    1. If $\langle B, w\rangle \in A_{\text{DFA}}$ (DFA B accepts the string w), M accepts $\langle B, w\rangle$
    2. If $\langle B, w\rangle \notin A_{\text{DFA}}$ (DFA B rejects the string w), M rejects $\langle B, w\rangle$
- We learn an algorithm how to simulate a DFA on an input string in Chapter 1
  - Turing machine can do the same

Decidable Languages

- The language A_{DFA}

$$A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ is a DFA that accepts input string } w\}$$

- An example of a Turing machine M is as follows:

$M =$ "On input $\langle B, w\rangle$, where B is a DFA and w is a string:

1. Simulate B on input w.
2. If the simulation ends in an accept state, *accept*. If it ends in a nonaccepting state, *reject*."

- Notes about the above TM
  - Input format $\langle B, w\rangle$ is the same as format of strings in A_{DFA}
  - The **where** clause filters out the following cases:
    - B is not a DFA
    - w is not a string

Decidable Languages

- An example of a Turing machine M is as follows:

  $M =$ "On input $\langle B, w \rangle$, where B is a DFA and w is a string:

  1. Simulate B on input w.
  2. If the simulation ends in an accept state, $accept$. If it ends in a nonaccepting state, $reject$."

- We do not know whether M is a decider for A_{DFA} yet
- We need to show that M is a decider for the language A_{DFA} by proving that
  1. If $\langle B, w \rangle \in A_{\text{DFA}}$, M accepts $\langle B, w \rangle$
  2. If $\langle B, w \rangle \notin A_{\text{DFA}}$, M rejects $\langle B, w \rangle$

Is M a Decider for the Language A_{DFA}

- Again, to show that the Turing machine M is a decider for the language A_{DFA}, we have to show that
  1. If $\langle B, w \rangle \in A_{\text{DFA}}$, M accepts w
  2. If $\langle B, w \rangle \notin A_{\text{DFA}}$, M rejects w
- Proofs:
  - Case 1: Assume that $\langle B, w \rangle \in A_{\text{DFA}}$. Since $\langle B, w \rangle \in A_{\text{DFA}}$, by the definition of A_{DFA}, B is a DFA that accepts the string w. Since B accepts w, by simulating B on input w in step 1, the simulation will end in an accept state. Since the simulation ends in an accept state, M accepts $\langle B, w \rangle$.
  - Case 2: Assume that $\langle B, w \rangle \notin A_{\text{DFA}}$. Since $\langle B, w \rangle \notin A_{\text{DFA}}$, by the definition of A_{DFA}, B is a DFA that rejects the string w. Since B rejects w, by simulating B on input w in step 1, the simulation will end in a non-accept state. Since the simulation ends in a non-accept state, M rejects $\langle B, w \rangle$.
- We just show that M accepts all strings in A_{DFA} and rejects all strings not in A_{DFA}
- M is a decider for A_{DFA}. Therefore, A_{DFA} is decidable

Prove that A_{DFA} is Decidable

Prove that

$$A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ is a DFA that accepts } w\}$$

is decidable.

Solution: Construct a TM M as follows:

$M =$"On input $\langle B, w\rangle$, where B is a DFA and w is a string:

1. Simulate B on input w.
2. If the simulation ends in an accept state, *accept*. If it ends in a nonaccepting state, *reject*."

Next, we need to prove that M decides A_{DFA}:

- Case 1: Assume that $\langle B, w\rangle \in A_{\text{DFA}}$. Since $\langle B, w\rangle \in A_{\text{DFA}}$, by the definition of A_{DFA}, B is a DFA that accepts the string w. Since B accepts w, by simulating B on input w in step 1, the simulation will end in an accept state. Since the simulation ends in an accept state, M accepts $\langle B, w\rangle$.
- Case 2: Assume that $\langle B, w\rangle \notin A_{\text{DFA}}$. Since $\langle B, w\rangle \notin A_{\text{DFA}}$, by the definition of A_{DFA}, B is a DFA that rejects the string w. Since B rejects w, by simulating B on input w in step 1, the simulation will end in a non-accept state. Since the simulation ends in a non-accept state, M rejects $\langle B, w\rangle$.

This shows that M is a decider for A_{DFA}. Therefore, A_{DFA} is decidable.

Decidable Languages

A_{NFA}

Let $A_{\text{NFA}} = \{\langle B, w\rangle \mid B \text{ is an NFA that accepts input string } w\}$. Show that A_{NFA} is decidable.

- Again, to show that A_{NFA} is decidable, we need to construct a TM N and prove the following:
  1. If $\langle B, w\rangle \in A_{\text{NFA}}$, N accepts $\langle B, w\rangle$
  2. If $\langle B, w\rangle \notin A_{\text{NFA}}$, N rejects $\langle B, w\rangle$
- As discussed earlier in Chapter 1, there is an algorithm that allows us convert an NFA B into an equivalent DFA C
  - Since there is an algorithm, a Turing machine that convert an NFA B into a equivalent DFA C where $L(B) = L(C)$
  - If NFA B accepts w, DFA C also accepts w
  - If DFA C accepts w, $\langle C, w\rangle \in A_{\text{DFA}}$
  - If $\langle C, w\rangle \in A_{\text{DFA}}$, TM M (decider for A_{DFA}) accept $\langle C, w\rangle$

Decidable Languages

- TM N can be constructed as follows:

  $N =$ "On input $\langle B, w \rangle$, where B is a NFA and w is a string:

  1. Convert NFA B to an equivalent DFA C, using the procedure for this conversion given in Theorem 1.39.
  2. Run TM M from Theorem 4.1 on input $\langle C, w \rangle$.
  3. If M accepts, *accept*; otherwise, *reject*."

- Notes about TM N
  - Recall that the language of TM M is A_{DFA}:

    $$A_{\text{DFA}} = \{\langle B, w \rangle \mid B \text{ is a DFA that accepts } w\}$$

  - TM N runs TM M on input $\langle C, w \rangle$ where
    - C is a DFA that it just constructed from NFA B
    - w is a string
    - M will accept or reject $\langle C, w \rangle$ depending on whether $\langle C, w \rangle$ in A_{DFA}
  - TM N runs TM M on input $\langle B, w \rangle$, M will always reject the input since B is an NFA

Prove that A_{NFA} is Decidable

Prove that $A_{\text{NFA}} = \{\langle B, w\rangle \mid B \text{ is an NFA that accepts } w\}$ is decidable.

Solution: Construct a TM N as follows:

$N =$"On input $\langle B, w\rangle$, where B is a NFA and w is a string:

1. Convert NFA B to an equivalent DFA C, using the procedure for this conversion given in Theorem 1.39.
2. Run TM M from Theorem 4.1 on input $\langle C, w\rangle$.
3. If M accepts $\langle C, w\rangle$, *accept*; otherwise, *reject*."

Next, we need to prove that N decides A_{NFA}:

1. Case 1: Assume that $\langle B, w\rangle \in A_{\text{NFA}}$. Since $\langle B, w\rangle \in A_{\text{NFA}}$, by the definition of the language A_{NFA}, B is an NFA that accepts the string w. By converting the NFA B into an equivalent DFA C ($L(B) = L(C)$), C also accepts the string w. Since C is a DFA that accepts the string w, $\langle C, w\rangle \in A_{\text{DFA}}$. Since M is a decider for A_{DFA}, by running M on input $\langle C, w\rangle$, M will accept $\langle C, w\rangle$ which will cause N to accept $\langle B, w\rangle$.
2. Case 2: Assume that $\langle B, w\rangle \notin A_{\text{NFA}}$, Since $\langle B, w\rangle \notin A_{\text{NFA}}$, by the definition of the language A_{NFA}, B is an NFA that rejects the string w. By converting the NFA B into an equivalent DFA C ($L(B) = L(C)$), C also rejects the string w. Since C is a DFA that rejects the string w, $\langle C, w\rangle \notin A_{\text{DFA}}$. Since M is a decider for A_{DFA}, by running M on input $\langle C, w\rangle$, M will reject $\langle C, w\rangle$ which will cause N to reject $\langle B, w\rangle$.

This shows that TM N is a decider for A_{NFA}. Therefore, A_{NFA} is decidable.

Decidable Language

A_{REX}

Let
$A_{\text{REX}} = \{\langle R, w\rangle \mid R \text{ is a regular expression that generates string } w\}$.
Show that A_{REX} is decidable.

- Again, to show that A_{REX} is decidable, we need to construct a TM P and prove the following:
  1. If $\langle R, w\rangle \in A_{\text{REX}}$, P accepts $\langle R, w\rangle$
  2. If $\langle R, w\rangle \notin A_{\text{REX}}$, P rejects $\langle R, w\rangle$
- Recall that we have an algorithm that generates an NFA A from a regular expression R where the language of A is expressed by R
  - Again, since there is an algorithm, we can have a Turing machine that convert a regular expression R into an equivalent NFA A

Prove that A_{REX} is Decidable

Prove that $A_{\text{REX}} = \{\langle R, w\rangle \mid R \text{ is a regular expression that generates string } w\}$ is decidable.

Solution: Construct a TM P as follows:

$P =$"On input $\langle R, w\rangle$, where R is a regular express and w is a string:

1. Convert regular expression R to an equivalent NFA A by using the procedure for this conversion given in Theorem 1.54.
2. Run TM N from Theorem 4.2 on input $\langle A, w\rangle$.
3. If N accepts $\langle A, w\rangle$, *accept*; if N rejects $\langle A, w\rangle$, *reject*."

Next, we need to prove that P decides A_{REX}:

1. Case 1: Assume that $\langle R, w\rangle \in A_{\text{REX}}$. Since $\langle R, w\rangle \in A_{\text{REX}}$, from the definition of A_{REX}, R is a regular expression that generates the string w. Since R is a regular expression that generates w, by converting R into an equivalent NFA A, A accepts w. Since NFA A accepts w, $\langle A, w\rangle \in A_{\text{NFA}}$. Since TM N is a decider for A_{NFA}, by running N on input $\langle A, w\rangle$, N will accept $\langle A, w\rangle$ which causes P to accept $\langle R, w\rangle$.
2. Case 2: Assume that $\langle R, w\rangle \notin A_{\text{REX}}$. Since $\langle R, w\rangle \notin A_{\text{REX}}$, from the definition of A_{REX}, R is a regular expression that does not generate the string w. Since R is a regular expression that does not generate w, by converting R into an equivalent NFA A, A rejects w. Since NFA A rejects w, $\langle A, w\rangle \notin A_{\text{NFA}}$. Since TM N is a decider for A_{NFA}, by running N on input $\langle A, w\rangle$, N will reject $\langle A, w\rangle$ which causes P to reject $\langle R, w\rangle$.

This shows that TM P is a decider for A_{REX}. Therefore, A_{REX} is decidable.